# MATE-SOCS: a Variable Topology Circuit-On-Chip Solver for Electric Circuits

Kyle Diller

v0.1

## SimScape Electrical, HDL Coder and FPGA modeling

One really nice feature of the MATLAB/Simulink suite is that it comes with code generation capability. In the automotive industry for example, it has become an unavoidable reference and Simulink designed controllers are present in most cars sold today.

Simulink code generation can be extended to FPGA devices with HDL Coder. But it is not so easy as it may seem… FPGA are really complex devices and come with many design constraints.

First of all, why using FPGA instead of CPU? Since electric system have much faster time constant than mechanical system for example, controller loop time is typically much smaller. Also, the latency of the HIL simulation must be kept very small so that the controller can reacts quickly to faults for example.

Short-circuit detection in motor drives is a good example. In the drive industry, it is common to verify the IGBT action of a motor inverter by monitoring the inverter input current. Indeed, each time an upper-IGBT starts or stop conducting, the DC current instantaneously changes. This is verified by the controller with a 1 MHz bandwidth typically. So, in HIL application, the latency must be of this order, including the Digital Input and Analog Output delays.

Then, by coding the electric system equations (ex: a motor drive) on the FPGA, it gets connected very close to the I/Os of the simulator, which are on the FPGA also. No more PCIe transfer delays between CPU and FPGA then.

But coding a DAE solver on FPGA is really hard. Especially if the time step objective is 1 micro-seconds. (There are many papers of FPGA solution at 10-15 us, but what is the purpose?!)

With Simscape Electrical and HDL Coder, we have to deal with some limitations. For the electric circuit to be deployable on an FPGA card, that is to limit the size of the FPGA design to fit into the FPGA, we need the circuit to:

- have a single linear mode of operation for the main circuit equations. This means no binary switches because each switch induces mode. For a circuit with $n$ switches, there are $2^n$ linear circuit modes.

- Non-linear model like machines must have a constant admittance as seen by the SimScape DAE solver. The non-linear part is typically modeled with delayed current source injections. It is common to have null admittance model for machine in many software (like SPS did): these are pure injection method, often prone to numerical instability.

In MATE-SOC, we deal with these limitations in clever ways:

Inverters: The MATE inverter have no binary switch inside. Thet are either Pejovic-style inverter (with power compensation) or Switching Function with High Impedance Capability (SFHIC) and are very accurate at time steps of 1 micro-seconds. SFHIC is over-sampling capable, meaning that it can compute the averaged voltage inverter from gate signals sampled at much higher frequencies (like 5-10 ns in FPGA-HIL applications).

Machines: MATE provides the users with constant admittance machine, much similar to PSCAD.

- An induction machine with fixed referential naturally provides a constant admittance for the DAE solver.
- A round rotor BLDC has a constant mutual inductance matrix and therefore a constant DAE admittance.
- A general PMSM with different $L_d$ and $L_q$ (like interior magnet SM) is presented a constant admittance to the DAE solver with the averaged DQ inductance. The difference part is injected with a delay. This approximation is much more stable than null admittance methods.

General switches (like breakers): they are represented by Pejovic switches. As these switches are not activated often, the Pejovic approximation is excellent in this case.

This results in motor drive and circuit that should be directly implementable in Speedgoat using HDL Coder.

## Model-based design workflow

Targeting FPGA can have some impact on the MBD workflow.

- Offline simulation of FPGA models can be painfully slow, if you simulate the models at the base sample time of the FPGA like 5-10 nanoseconds
- Some companies do not allow offline simulation but only on-FPGA simulation, which is really fast but lack common debugging methods like breakpoint; variable examination is tedious (when possible).
- FPGA model limitations can force user to change model (like PMSM model) between offline simulation and on-FPGA simulation, which decrease the trustworthiness of the complete test cycle.

## FPGA model sample time is not FPGA clock.

It is also important to be aware that in FPGA implementation, the simulation model sample time (with an objective of 1 micro-seconds or less typically) is not the same thing as the FPGA clock or FPGA sample time (typically 2-5 nanoseconds). Awareness of this fact can help alleviate some of the FPGA testing pains.

Let's take a simple Euler Integrator for example. In Simulink, the gain 'Ts' and the following adder are computed in an instantaneous sequence; this is called a Direct Feedthrough path in Simulink terminology.

And at each model sample time, the delay block is updated with the integration sum. Easy. The Model Sampling Clock is implicit in Simulink.

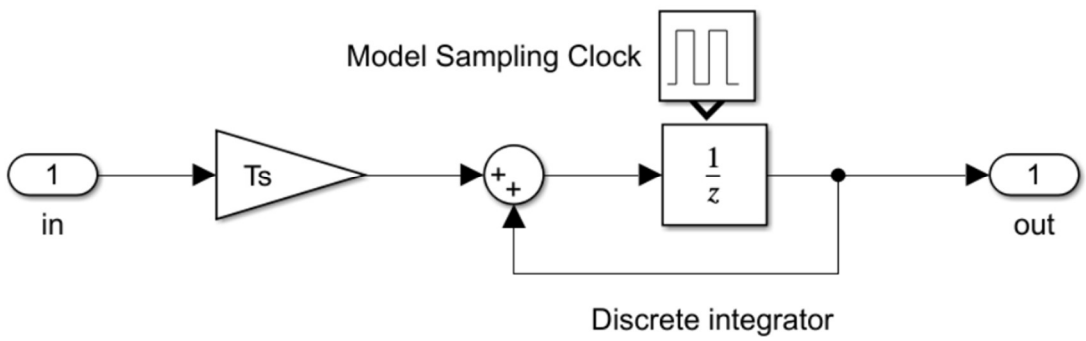



*Fig. 1 Integrator in Simulink*

On FPGA however, each operation takes some FPGA sample time to be completed. Typically latency is 2 or 3, depending on type (integer, float) or precision (single, double). Even a simple signal can have a latency if the user decides it!

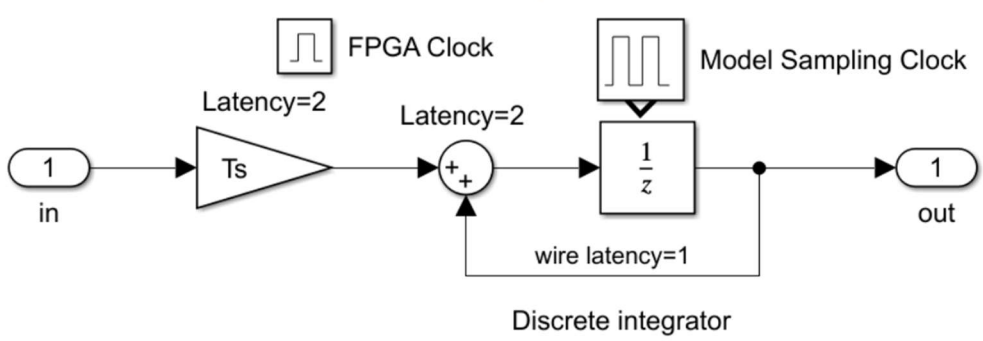



*Fig. 2 Integrator in Simulink with FPGA latencies*

What is confusing here is that:

1) the FPGA clock is not the model sample time
2) Direct Feedthrough operation must still be completed but now they take some time to complete (Latency=1 means 1 FPGA clock)

In the case above the model needs 4 FPGA clock time step to complete the integrator operation. In real model this minimum path is much higher (corresponding to higher operation complexity) and is precisely what determines the minimum model sample time (again, not the same as the FPGA clock).

> *The good side of this dual sampling method is that during simulation phase, you can temporarily set the FPGA operators latency to zero and simulate the model at its base sample rate without affecting the accuracy. Simulation of a complex model at 1 micro-seconds (with provisional zero latency option) will be much faster than doing it at 10 nanoseconds (with all latency included). Both results will be equal because in each case, all computations, as determined by the direct feedthrough ordering, are completed in the same manner.*

To summarize the FPGA design problematic, when designing a Simulink model for FPGA, there are several clock and timing constraints:

1- You have the sample time of the model (let's say a DAE solver). This is the same sample time of the model in regular Simulink. Let's aim Ts=1 us for this sample time.
2- You have the FPGA clock for the execution on the FPGA chip. For a 200 MHz FPGA, this clock is therefore Ts_fpga= 5 nanoseconds.

This ratio Ts/Ts_fpga=200 sets the limit for the total latency for FPGA operators between states of the model. These states will be Unit Delays (Ts=1us) blocks in Simulink, typically.

You also have the FPGA timing latency which is different. During FPGA design and synthesis, the user can select the latency of each basic FPGA operator. If many blocks are connected to the same output of a block (this is called fan-out) the propagation latency will increase do to the chip line capacitance and the physical design will not meet timing requirements and therefore will fail. Solution is to decrease fan-out or to allow more latency for each operator to complete.

Below in Fig. 3 is a simplified MATE-SOCS structure. So, let's say that you have one Simulink state that is updated by a Matrix product, with a feedback at the input. This could represent an inductance model discretized with the Backward Euler method connected to a nodal admittance matrix for example.

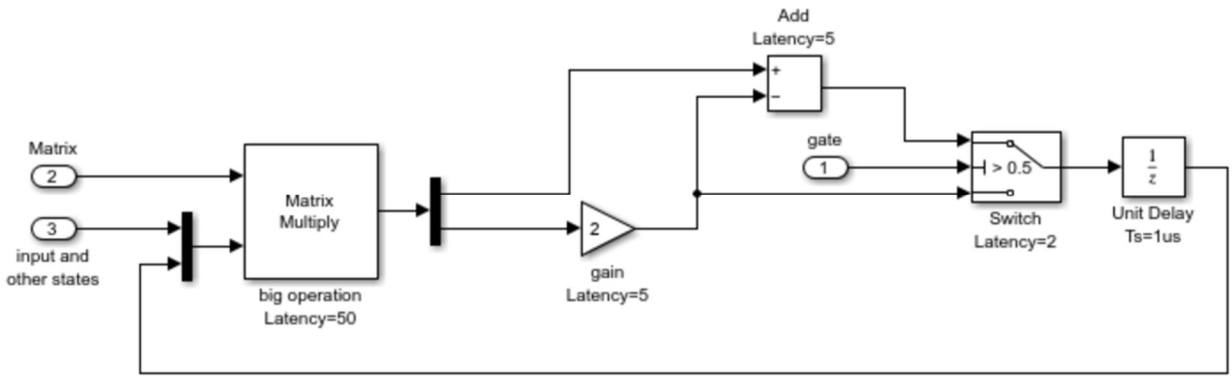



*Fig. 3 Solver structure with latencies*

One can observe that we need Ts_fpga to be at least 62 times smaller than Ts (the longest path =50 +5+ 5+ 2) for the model to work correctly. So if the desired model sample time equals 1 us, the FPGA must run 62 time faster (62 MHz, since 1 us corresponds to 1 MHz). This is easy in year 2025.

But what if we increase the size of the matrix operation? Let's say the worst latency=200 for example, then the FPGA must run at 200 MHz (Ts_fpga=5ns), which is less obvious but not difficult

FPGA synthesis timing issues are driving this total latency to increase, because it takes more operators and operations to make bigger matrix multiplications.

## The MATE Circuit-On-Chip Solver (MATE-SOCS)

Considering the long FPGA Place&Route (i.e. FPGA compilation) times of today's mammoth FPGA chips, we aim to provide a SOC solver than can simulate a variety of electric circuit with:

1- No need to recompile the FPGA bitstream for changes in electrical topology and parameters.
2- Aims 1 micro-seconds of sampling time, to fit well with 1 Msamples/s Analog-Outputs.
3- A straightforward design

Let there be a D matrix and empty ABC matrices representing the input-output relationship of a discretized circuit connecting various RLC and switching elements. We have the Pejovic switch. When, the gate is (always) ON, it simulates an inductor, when it is (always) OFF, a capacitor. In all 3 bases, the elements only need the voltage input and the local history value. Energy-compensated inverters are just 2 Pejovic switches with some links.

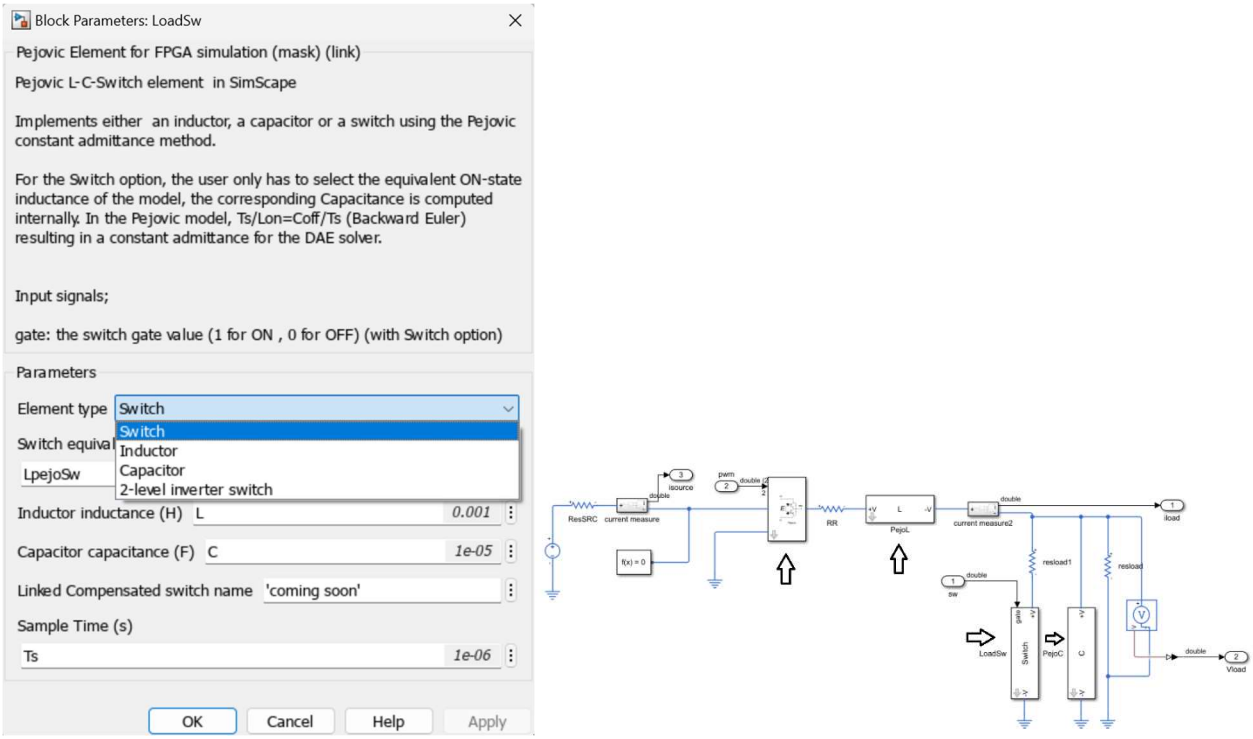



*Fig. 4 The Pejovic Element mask and usage in a Simscape model*

State-less elements like Resistor and ideal transformer are taken care in the composition of the D matrix.

So using this 'Pejovic Element' to represent L, C, switches, and inverters in a standard SimScape model, we can actually simulate the circuit in offline mode!

<u>Next step: Extract the data for MATE-SOCS.</u>

This is done by running the MakeMate2 script on the Simulink model.

[MATE, MATERT,blk]= MakeMate2(myMATEmodel 1);

MakeMate2 (with myMATEmodel being the subsystem containing the off-line Pejovic circuit) will produce all the topological and parameter information to run a discrete state-space simulation. The 'Pejovic Element' containing the actual L-C-switch states, running MakeMate2 will produce a D state-space matrix with empty ABC ones.

<u>Some special treatment is done by MateConvertForFPGA()</u>

[MATE_SOC]=MateConvertForFPGA(MATE)

Indeed, to run a variable topology model on a fixed FPGA bitstream, inverters, single switches, inductors and capacitor inputs and outputs have to be re-ordered in a constant way. Parameter list will also be aware of this order of elements. The nice thing about the Pejovic Elements is that they all output a so-called History value and read a voltage value, so the first inputs and outputs of the system (represented by the D matrix) are the input-outputs of the Pejovic elements.

Inverters typically work on a duty-cycle basis, so PWM modulator is included in the SOC, followed by switch gates, all Pejovic admittance values, Energy-compensated inverter indexes.

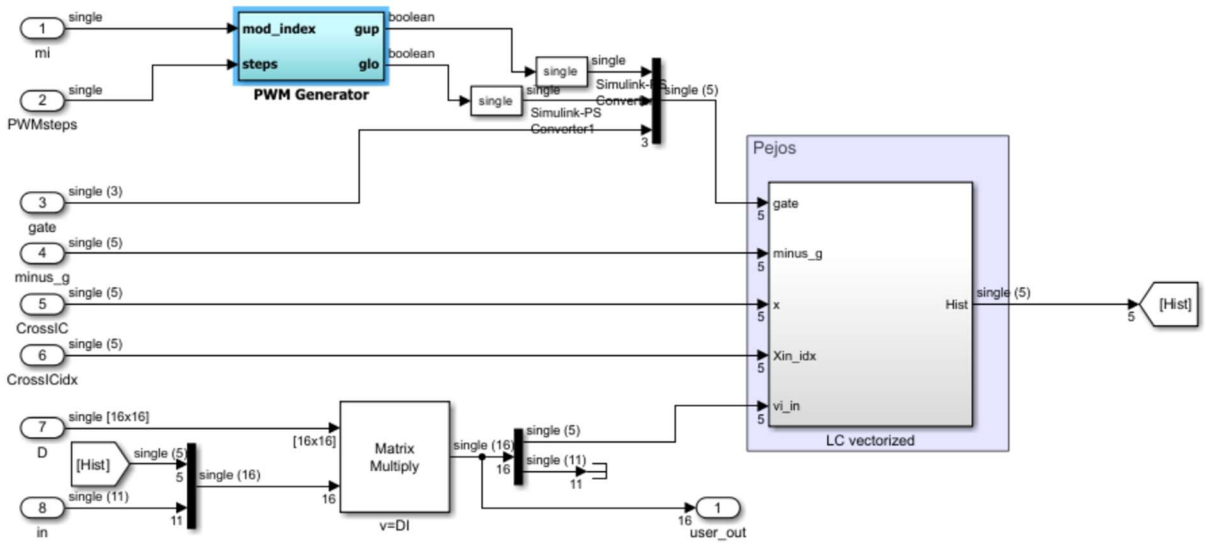



*Fig. 5 MATE-SOCS (gen1) structure*

For the Pejovic element itself, it implements the L/C equations discretized by Backward Euler method and contain their own states (which is why ABC are empty). The lower part of the model is for the energy compensation. Note that the design is done so to adapt to different topology without re-wiring, in preparation for HDL Coder. For example, the lower Simulink Selector of energy compensation index is implemented with user-input indexes.

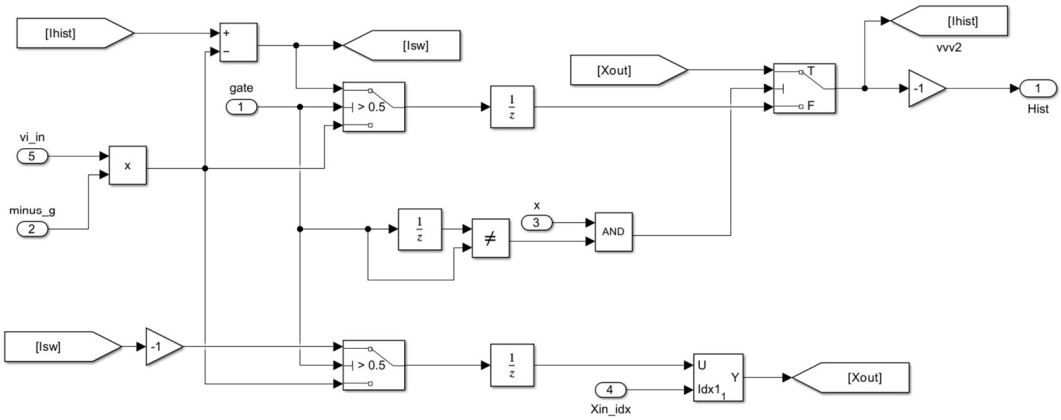
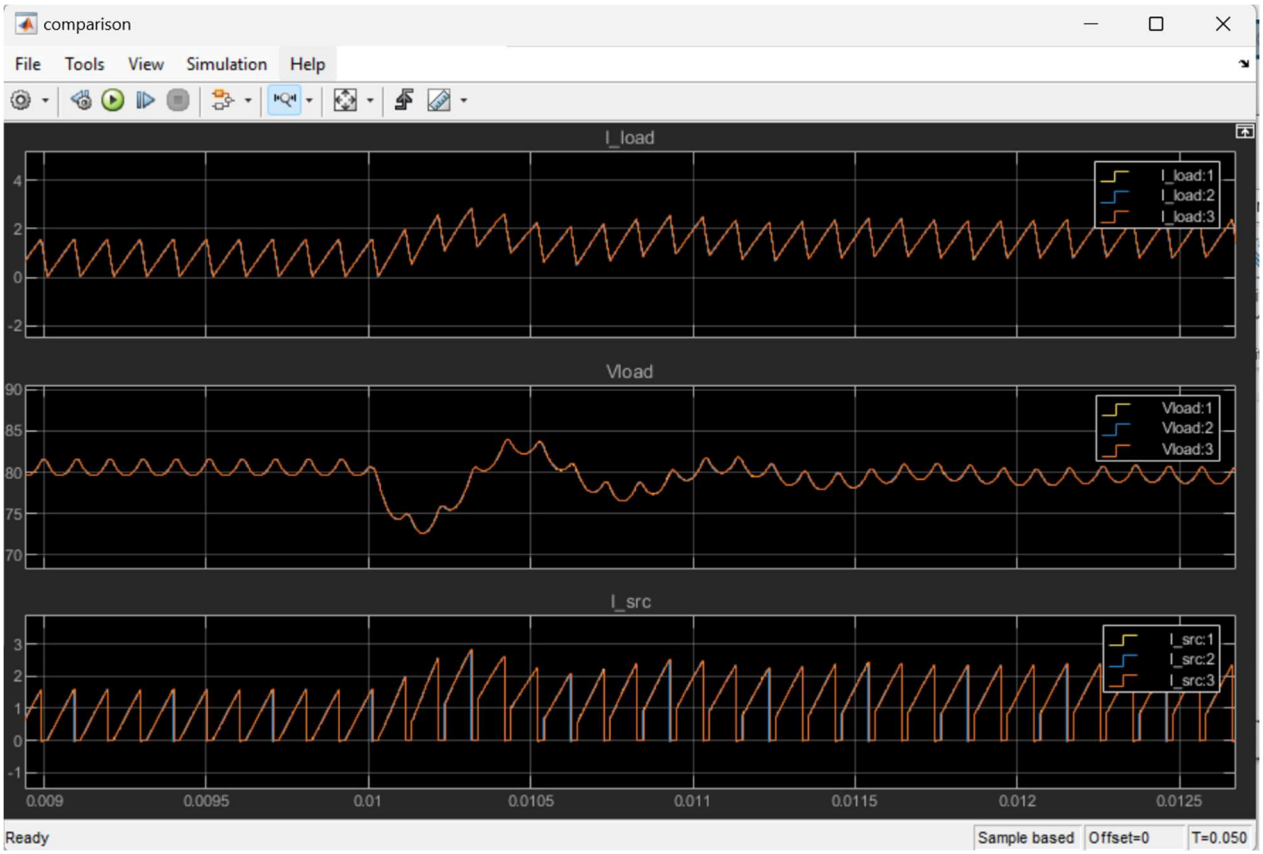



*Fig. 6 Pejovic Element internal structure*

Then, one can easily compare 3 things offline:

- The Simscape simulation with ALL native Simscape RLC, switch elements.
- The Simscape simulation with Pejovic Elements, using the Simscape solver
- The SOC simulation (without any Simscape elements)

As expected, and required, the 3 simulation give identical results below.




## Current dimension limits of MATE-SOC

MAX_SWITCHES=8;

MAX_OUTPUTS=16;

MAX_INPUTS=16;

(including MAX_SWITCHES, so there are really 8 free inputs/outputs, the first 8 are for the switches)

<u>Supported elements</u>: R, L, C, ideal switch, ideal 2-level inverters (the PMW modulators have no dead-time)